

Hao (Howard) Guo

AI Engineer · LLM applications, RAG and inference

Jersey City, NJ · howardguoui@gmail.com · linkedin.com/in/hao-guo-918690126 · github.com/howardguoui · howardguoui.github.io

SUMMARY

Vice President and software engineer at Bank of America with 9+ years building data-heavy applications for banking and enterprise software. Lead AI-driven diagnostics and remediation for enterprise device enrollment, and build and measure generative AI systems end to end: RAG with hybrid vector search and RAGAS evaluation, LLM serving with vLLM and llama.cpp, and GPU job pipelines.

SKILLS

- **AI & LLMs:** RAG, LLM evaluation (RAGAS, LLM-as-judge), embeddings, prompt engineering, structured outputs, AI agents and tool calling (MCP, Microsoft Agent Framework), Claude API, LangChain, Ollama, Whisper
- **Inference & MLOps:** vLLM, llama.cpp, quantization (AWQ, GGUF), KV cache tuning, CUDA and NVML profiling, PyTorch, Docker, Prometheus, GitHub Actions
- **Data & retrieval:** PostgreSQL, pgvector, full-text and hybrid search (RRF), cross-encoder reranking, MySQL, MongoDB, DynamoDB, pandas, NumPy
- **Software:** Python, Java, TypeScript, FastAPI, Flask, Celery, Redis, Node.js, React, Redux, Angular, Next.js, WebSockets, pytest, Jest

EXPERIENCE

Bank of America · *Vice President, Software Engineer* Oct 2020 – Present

- Led the integration of AI-driven diagnostics and remediation into device enrollment services: pattern detection and anomaly analysis over enrollment issues that recommend a fix automatically, cutting weekly manual investigation by 25%.
- Applied generative AI to enrollment, duplication and remediation data to surface recurring issues, speeding root-cause analysis and reducing repeat incidents.
- Partnered with engineering and product teams to ship AI-powered issue triage for enterprise device management that triages tickets automatically and creates Jira issues without manual entry.
- Build React and TypeScript dashboards for a trading platform that streams real-time market data to Global Banking & Markets traders and finance users, helping them manage market risk and reduce mark adjustments.

BNY Mellon · *Software Engineer* Jan 2020 – Oct 2020

- Built React and Java features and Adobe Experience Manager (AEM 6.5) content workflows for BNY Mellon.com and Pershing.com.

Bank of America · *Software Engineer, GBAM Risk Strategy* Oct 2018 – Jan 2020

- Built reporting applications (Brisk, Inbox, P&L) for the Reporting and Analytics team, serving risk executives' periodic credit reporting, with React, Redux, OpenFin and WebSockets, tested with Jest and built through Jenkins.
- Maintained Python Flask services on Quartz, Bank of America's cross-asset pricing and risk platform, and built regulatory and trade-metrics reporting for 300+ internal users.
- Coordinated offshore engineering teams in India, Hong Kong, Japan, Brazil and the UK.

ADP · *Software Engineer* Jan 2017 – Sep 2018

- Migrated the Global Cloud Connect data-integration application from AngularJS 1.5 to Angular 6 and TypeScript.
- Built shared Angular modules reused across the product and raised unit and end-to-end test coverage with Karma, Jasmine and Protractor; worked with Docker and AWS.

Appriss · *Software Engineer* Sep 2016 – Dec 2016

- Built Angular 2 and Node.js/Express applications backed by DynamoDB for VINELINK, a national victim-notification service.

- Built features for the Smart Order Management System.

SELECTED AI PROJECTS

Local Inference Lab · Python, vLLM, llama.cpp, Ollama, NVML, Prometheus, FastMCP, Docker

- Benchmarked vLLM, llama.cpp and Ollama serving the same 7B model at 4-bit on a 16 GB GPU with an async streaming load generator (1 to 48 concurrent streams): at 16 streams vLLM sustained 1,210 tokens/s vs 527 for llama.cpp and 99 for Ollama, with a median time to first token of 0.9 s vs 37 s.
- Measured FP8 against FP16 KV cache under a 4k-token, 48-stream load: 1.7× the KV capacity (272k vs 158k tokens), 21% more throughput and no request preemptions (vs 6).
- Built a VRAM planner that predicts vLLM's KV cache allocation from model shape and sizes llama.cpp partial offload from GGUF tensor sizes, and exposed planning and benchmarks to AI agents as an MCP server.

Filings RAG · Python, FastAPI, PostgreSQL + pgvector, fastembed, Claude API / Ollama / vLLM, RAGAS, Docker

- Built RAG over SEC 10-K filings with hybrid retrieval in one Postgres query (pgvector HNSW plus full-text search, fused with Reciprocal Rank Fusion) and optional cross-encoder reranking: hybrid retrieval reached hit rate@6 of 84% and MRR 0.83, against 63% and 0.39 for a random ranking.
- Built an LLM evaluation suite on 36 labeled questions (32 answerable, 4 unanswerable) with retrieval metrics and RAGAS, graded by a local 8B judge: 97% faithfulness, 91% answer relevancy, citations in every answer, and correct refusals on 4/4 unanswerable questions with no false refusals.
- Shipped it as a Dockerized FastAPI service with pooled database connections, rate limits and a daily cost cap, CI-tested against a real pgvector database.

Workflow Copilot · Python, Microsoft Agent Framework, Microsoft 365 Agents SDK, Ollama, WebLLM, JavaScript

- Built a compiler that turns an Obsidian workflow note (steps, linked tools, rules) into a Microsoft Agent Framework agent served through the Microsoft 365 Agents SDK endpoint that Agents Playground and Teams use, on any OpenAI-compatible model.
- Debugged agent reliability on real runs: Qwen3 8B skipped tool calls and answered from memory until note-level instructions and greedy decoding fixed it; a browser version on Qwen2.5 3B (WebLLM, WebGPU) forces a tool call first.

AI Video Factory · Python, FastAPI, Celery + Redis, faster-whisper, Ollama, Qwen3-TTS, FFmpeg, Next.js

- Orchestrated a four-stage speech-to-speech translation pipeline (Whisper ASR, LLM translation, voice-cloned TTS, FFmpeg) on one 16 GB GPU, loading each model only when needed; switching to Whisper large-v3-turbo made transcription 3.5× faster (17.1× vs 4.9× real time) at near-identical accuracy (9.1% vs 9.0% CER).
- Built a quality benchmark on Google FLEURS: 9.1% character error rate at 17.1× real time, and chrF++ 52.3 for Chinese-to-English translation with a local 8B model.
- Isolated Whisper in a subprocess per job so its VRAM is reclaimed on exit, and queued jobs through Celery with late acknowledgement so a crashed worker's job re-runs.

Agent Office · Claude Code hooks, WebSockets, MCP

- Built real-time observability for AI coding agents, streaming agent activity from Claude Code hooks over WebSockets, with an MCP server.

EDUCATION

M.S. Mathematics (Computer Science concentration) · North Carolina Central University

Dec 2018

- Research assistant on scholarship, and teaching assistant. Coursework: algorithms, data structures, big data systems, computer vision.